

StegaVault Simplified VTU Build Spec

StegaVault — Simplified Build Spec

VTU 7th Sem Major Project • MERN Stack

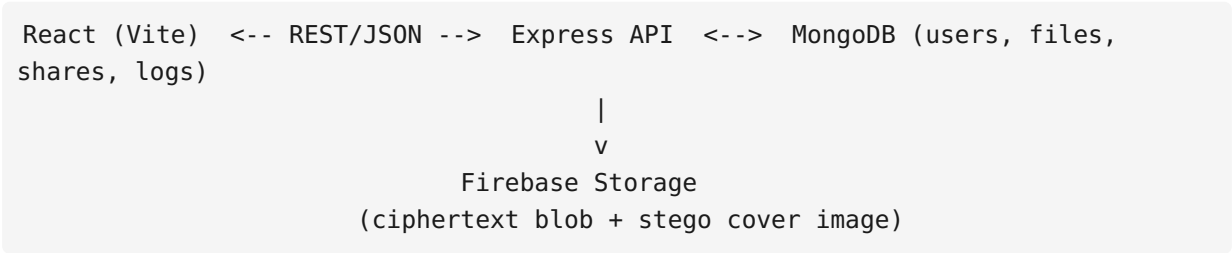
A right-sized rebuild of your Phase-I project (“Secure Cloud Storage with Encryption & Data Integrity Mechanisms”). Keeps every concept an examiner will want to see — AES encryption, LSB steganography, graphical password — while cutting the production infrastructure that isn’t needed for a student project.

1. Tech Stack

Layer	Technology	Why
Frontend	React (Vite) + plain CSS or Tailwind	You already know React
Backend	Node.js + Express	Simple, no framework overhead like NestJS
Database	MongoDB + Mongoose	No migrations, flexible schema, fast to set up
File storage	Firebase Storage (or Cloudinary)	Free tier, no AWS account/IAM setup
Auth	JWT (jsonwebtoken) + bcrypt	Standard, well-documented, no refresh-rotation complexity
Encryption	Node <code>crypto</code> module, AES-256-GCM	Built into Node, no extra dependency
Steganography	<code>jimp</code> (pure JS image library)	Pixel-level access for LSB, no native binary deps to install
File upload	<code>multer</code>	Standard Express file upload middleware

No Docker, no Redis, no CI/CD pipeline, no OAuth (unless you want to add Google login as a stretch goal later). Everything runs with `npm run dev` on both client and server.

2. Architecture



Difference from the original zero-knowledge design: **encryption happens on the server**, not in the browser. The client sends the raw file over HTTPS; the server encrypts it, embeds the key in a cover image, uploads both to Firebase, and never stores the plaintext file or raw key on disk. This is the same trust model your Phase-I report used — server does the work — just with a stronger

cipher (AES-GCM instead of CBC) and cloud storage instead of local disk.

(If you want to attempt true client-side/zero-knowledge encryption as a stretch goal once the basic version works, that's a good "advanced feature" to add in the last 1-2 weeks — but don't start there.)

3. Database Schema (MongoDB / Mongoose)

User

```
email, passwordHash (bcrypt), displayName,
graphicalPasswordHash (bcrypt of ordered image sequence, e.g. "3,17,8,25,11"),
role: 'user' | 'admin',
storageUsedBytes, storageQuotaBytes,
failedLoginCount, lockedUntil,
createdAt
```

File

```
owner (ref User), originalFilename, mimeType, sizeBytes,
storageUrlEncrypted (Firebase URL for ciphertext),
storageUrlStego (Firebase URL for cover image with hidden key),
iv (base64), authTag (base64), checksum (sha256),
isDeleted, deletedAt,
createdAt
```

Share

```
file (ref File), owner (ref User), recipientEmail,
tokenHash (sha256 of a random token — don't store the raw token),
permission: 'view' | 'download',
expiresAt, revokedAt, createdAt
```

AccessLog

```
user (ref User, nullable), action (login_success, login_failed, upload,
download, share_create, ...),
resourceType, resourceId, ipAddress, createdAt
```

That's 4 collections — simple enough to explain in a viva, complex enough to be a real relational-ish design (even in Mongo, you're using refs deliberately, which you can justify).

4. Core Feature: Encryption + Steganography Pipeline

Upload 1. Client sends the raw file to `POST /api/files/upload` (multer handles multipart). 2. Server generates a random 256-bit AES key and 96-bit IV (`crypto.randomBytes`). 3. Server encrypts the file: `crypto.createCipheriv('aes-256-gcm', key, iv)` — GCM mode gives you a built-in auth tag, so “Data Integrity Mechanisms” in your title is literally true: any tampering is detected on decrypt. 4. Server takes a cover PNG (either a fixed default image or one the user uploads) and uses `jump` to embed the 256 key bits into the least-significant bits of the pixel data — the same LSB scheme from your Phase-I report. 5. Server uploads the ciphertext blob and the stego-PNG to Firebase Storage. 6. Server saves the IV, auth tag, checksum and both storage URLs in MongoDB. The raw AES key is **never stored anywhere** — it only exists transiently in server memory during this request, and permanently inside the stego image's pixels.

Download 1. GET /api/files/:id/download — server checks ownership/share validity. 2. Server fetches the stego-PNG from Firebase, extracts the 256 key bits from the LSBs with `jmp`, reconstructs the AES key. 3. Server fetches the ciphertext, decrypts with `crypto.createDecipheriv`. If GCM’s auth tag check fails (tampering), decryption throws — no corrupted output is ever returned. 4. Server streams the decrypted file back to the client as a download.

This is the heart of your project and the part to build first (after basic auth), since everything else — sharing, dashboard, logs — is routine CRUD around it.

5. Auth Flow (kept from your original design)

1. **Register:** email + password (bcrypt hash) + set up the graphical password (pick 5 images from a grid, in order — store only the bcrypt hash of the ordered sequence, e.g. hash of "3,17,8,25,11").
2. **Login step 1:** email + password → server returns a short-lived “ticket” (not a full session yet).
3. **Login step 2:** server sends a freshly randomized image grid; user clicks their 5 images in order; server hashes the clicked sequence and compares to the stored hash. On match, issues a JWT.
4. Lock the account for 15 minutes after 5 failed attempts (on either factor) — simple to implement with `failedLoginCount` and `lockedUntil` fields on the User document.

Keep JWT simple: one token, ~1 hour expiry, sent in an `Authorization: Bearer` header. Skip refresh-token rotation — it’s a lot of extra complexity for very little demo value.

6. Backend API Surface

Method	Path	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Step 1: verify password, return ticket
GET	/api/auth/graphical-challenge	Get randomized image grid
POST	/api/auth/graphical-verify	Step 2: verify sequence, issue JWT
POST	/api/files/upload	Encrypt, hide key, upload to Firebase
GET	/api/files	List current user’s files
GET	/api/files/:id/download	Reconstruct key, decrypt, stream file
DELETE	/api/files/:id	Soft delete
POST	/api/shares	Create a share link for a file
GET	/api/shares/:token	Resolve a share link (public)
GET	/api/logs/me	Current user’s activity log
GET	/api/admin/users	(admin) list users — stretch goal

That’s roughly 12 endpoints — a manageable Express app, not a multi-module NestJS project.

7. Frontend Pages

1. **Landing** — short pitch, sign up / log in buttons
2. **Register** — email/password + graphical password setup
3. **Login (2 steps)** — password, then image grid
4. **Dashboard** — file list, upload button, storage used
5. **Upload flow** — progress indicator: “Encrypting → Hiding key → Uploading → Done”
6. **File detail** — download, share, delete
7. **Shared with me** — files others shared with you
8. **Activity log** — table of your own actions
9. *(Optional)* Admin page — list users, view global logs

9 pages is plenty for a strong demo without becoming unmanageable.

8. Build Roadmap (Realistic for a Student Timeline)

Phase	Time	Goal
0 — Setup	2-3 days	React + Express + MongoDB connected, basic “hello world” round trip
1 — Auth	1 week	Register, password login, graphical password setup + verify, JWT
2 — Encryption + Stego pipeline	1.5-2 weeks	Upload → encrypt → hide key → Firebase; Download → extract key → decrypt. Build and test this with Postman before touching the UI.
3 — Dashboard + file UI	1 week	File list, upload widget with progress states, download, delete
4 — Sharing + Activity log	1 week	Share links, access logs wired into every action
5 — Admin panel + polish	3-5 days	Optional admin page, styling pass, error handling
6 — Report + deployment	1 week	Deploy frontend on Vercel, backend on Render (both free tier), write up the report

Total: ~6-7 weeks part-time. Phase 2 is the one to protect time for — it’s the technical core of your project and the part your report and viva will focus on.

9. What Makes This Still a “Real” Major Project

Even simplified, you’re demonstrating: - Symmetric encryption with authenticated integrity (AES-GCM) - A non-trivial data-hiding technique (LSB steganography) implemented from scratch - A genuine second authentication factor with secure hash-only storage - Secure file sharing with hashed tokens and expiry - A full audit trail - A working full-stack app with a real database and cloud storage

That’s a legitimate, defensible VTU major project scope — the earlier document was written for someone planning to run this as an actual product, which is a different (much bigger) job.

